

Viacheslav Androsov

Frontend Developer (React / Vue / TypeScript)

Open to remote work

bandrone.man@gmail.com · t.me/awaitMeBro · portfolio

SUMMARY

Frontend developer with 4+ years of commercial experience, specialising in Vue 3, React and TypeScript. I build optimised, high-performance applications, design frontend architecture and put legacy code back in order. Experienced in fullstack development with Node.js. I work with Claude Code and AI agents daily. Ready to take on complex problems and improve the user experience.

CORE SKILLS

Frontend: TypeScript, React, React Native (Expo), Vue 3, Next.js, Redux Toolkit, Zustand, Pinia, TanStack Query / Table, React Hook Form

UI / Styling: Tailwind CSS, shadcn/ui (Radix), SCSS, Storybook, responsive layouts

Quality / Tooling: Vite, Zod / Yup, OpenAPI, MSW, Vitest, Jest, React Testing Library, Playwright, FSD, CI/CD, Git

Web3: ethers.js, web3.js, MetaMask, WalletConnect, smart-contract integration

AI: AI-assisted development — Claude Code, agentic pipelines, MCP

WORK EXPERIENCE

Syncra — Frontend Developer (Vue) *Apr 2025 — present*

B2B payment gateway for online stores and services, about 40k payments a day. Several teams worked on the project; mine had 2 frontend developers, 3 backend developers, a business analyst, a QA engineer and a team lead. Scrum process, new features shipped in planned releases.

Achievements:

- Caught a **payment form crash on some Android devices** through Sentry: an old browser engine could not parse the bundle and copying payment details failed silently. Added fallback paths and a build for older browsers — **payment-step conversion on Android rose from ~62% to ~81%**.
- Designed the history module for the business-process editor — 40+ operation types, including moving blocks between diagrams (pools, nested subprocesses).
- Sped the editor up from ~25 to a stable 60 FPS on diagrams of 100+ blocks: removed the idle repaint loop, took scene geometry out from under Vue reactivity and rebuilt the frame so that measuring and painting no longer interleave.
- Brought **Claude Code with MCP servers** into the team and taught colleagues to work with AI agents — tests, reviews and documentation began taking noticeably less time.
- Built a **reusable DataGrid on TanStack Table** used on 20+ pages of the app: filtering, sorting, pagination and drag & drop column setup. Added live transaction statuses over WebSocket: an update repaints one row, not the whole table — **500+ transactions update without dropped frames**.
- Introduced **in-app user feedback collection**: merchant requests were gathered into statistics, and the team picked them up faster.
- Moved block contents onto Vue components over the canvas: forms inside blocks are assembled from a description supplied by the backend, so a new block type plugs in with no hand-written markup on the frontend.
- Integrated the editor's AI assistant into the frontend: the dialogue, a preview of the suggested steps on the diagram, applying them through the shared action buffer. In parallel, set up how the team works with Claude Code — test generation, change review, keeping architecture documentation alive.

- Introduced dynamic validation for financial forms: the set of fields depends on the payout method and the currency — fewer input errors and support tickets.

Responsibilities:

- Building the merchant portal and process editor interfaces
- Refactoring code and moving modules onto FSD
- Code review
- Onboarding new developers
- Fixing bugs, including from Sentry reports
- Maintaining and evolving the frontend infrastructure: GitLab CI, build, linters

Stack: *Vue 3, TypeScript, Pinia, TanStack Query / Table, VeeValidate + Zod, shadcn-vue, Tailwind CSS, SVG/DOM scene engine, WebSocket, Keycloak, Sentry, Vite, Vitest, FSD.*

MGS LLC — Frontend Developer (React, React Native) *Mar 2024 — Apr 2025*

Housing-management ecosystem: a dashboard for management companies and a mobile app for residents. A team of about 10: 3 frontend developers on the web, me on the mobile app, 3–4 backend developers, a designer, a QA engineer and a product manager. Scrum with two-week sprints: the web shipped at the end of each sprint, the mobile app as features were ready.

Achievements:

- Built from scratch a mobile app for residents and management-company staff on React Native: requests, payments, documents, push notifications.
- Took part in moving the legacy web app onto FSD architecture without pausing development. New developers started carrying their first task through to release in days instead of weeks.
- Initiated and introduced generation of a typed API client from the OpenAPI schema (Orval) into the team's workflow: types stopped drifting from the backend, manual synchronisation disappeared entirely, and contract breaks show up at build time, not in production.
- Sped the web app up: split the bundle by routes, added request caching and removed extra re-renders — the initial load and page transitions became noticeably faster.
- Built a permission system on CASL: role rules live in one place and components simply ask "is this action allowed" — a new role is added by editing one file rather than by checks all over the code.
- Set up API mocks on MSW — the frontend stopped waiting for the backend to be ready, and the same mocks were reused in tests.
- Wrote **the project documentation from scratch in Markdown and Confluence** — new developers got up to speed faster.

Responsibilities:

- Building the web dashboard for management companies and the React Native mobile app
- Taking part in refactoring the project onto a clean FSD-like architecture
- Code review
- Onboarding new developers
- Fixing bugs in the web and mobile apps
- Maintaining and evolving the infrastructure: API client generation from OpenAPI, MSW mocks, CI/CD

Stack: *React, React Native (Expo), TypeScript, Zustand, TanStack Query, React Hook Form + Yup, CASL, OpenAPI / Orval, Tailwind CSS / NativeWind, MSW, Vite.*

BabyDoge — Frontend Developer *Jun 2022 — Feb 2024*

Web3 ecosystem on BNB Chain: a decentralised exchange, an NFT platform and GameFi projects — babydoge.com. A team of 2–3 frontend developers, a backend developer, a designer and a project manager. Kanban, releases shipped against product milestones.

Achievements:

- Joined as a junior; within six months **owned exchange modules — wallets, swap, portfolio**, and by the end shipped features from mockup to production and reviewed the second frontend developer.
- Built interfaces for a decentralised exchange on BNB Chain: trading with charts (TradingView, ApexCharts), portfolio, airdrops.

- Integrated **MetaMask and WalletConnect** through ethers.js: sign-in by message signature, network switching, and a transaction UI that shows both phases — signing and waiting for the block — and survives a page reload.
- Showed blockchain data in real time — prices, liquidity, yield — over WebSocket, with no page reloads.
- Built modules of an NFT platform with a revenue-sharing model (Stake / Trade / Earn).
- Built promo landing pages with GSAP animations to bring users into BabyDoge products.

Responsibilities:

- Building the exchange, NFT platform and promo landing page interfaces
- Refactoring exchange modules: wallets, swap, portfolio
- Fixing bugs, including in wallet and transaction flows
- Maintaining the frontend build and environment

Stack: *Vue 3, TypeScript, Pinia / Vuex, TanStack Query, vee-validate + Yup, Vite, SCSS / Tailwind CSS, ethers.js, web3.js, MetaMask, WalletConnect, ApexCharts, GSAP.*

PRODUCT

FoodIQ — pet project in production

foodiq.space

AI nutrition tracker: a meal is described in plain text, calories and macros are worked out automatically. Small team: a backend developer, an analyst and me on the whole frontend; the web app is live, the mobile app is heading for RuStore, the App Store and Google Play

- Designed the **FSD architecture** for the Next.js 16 web app and the React Native app.
- Set up **contract-first work with the backend**: we design the OpenAPI schema together, then frontend and backend build in parallel, with client types and hooks generated from the schema. **120+ endpoints** connected this way across the web and mobile apps.
- Built the **AI parsing UI streamed over SSE**: items appear as they are ready, a dropped connection does not break the result, and ambiguous matches are flagged honestly rather than guessed.
- Built a **dashboard builder with 16 widgets** and drag-and-drop: the layout is stored on the server and survives older client versions.
- **Halved the entry bundle — from 2.1 MB to 1 MB** by splitting the API client and lazy-loading heavy modules.

Stack: *Next.js 16 (App Router), React 19, React Native (Expo), TypeScript, NestJS, Tailwind CSS v4, shadcn/ui (Radix), TanStack Query, Zustand, React Hook Form + Zod, OpenAPI, Recharts, Vitest, Playwright, Storybook, FSD.*

EDUCATION

Higher education, Siberian Federal University (SibFU), 2014